9.1 Overview of term-level search

Term-level search is structured: queries return results in exact matches. They search for structured data such as dates, numbers, and ranges. With this type of search, we don't care how well the results match (how well documents correspond to the query), just that the query returns data if the query is matched. Hence, we do not expect a relevancy score associated with the results of a term-level search.

A term-level search produces a yes or no binary option similar to a database's `WHERE` clause. Query results are fetched if the condition is met; otherwise, the query doesn't return any results.

Although documents have scores associated with them, the scores don't matter. Documents are returned if they match the query, but not with relevance. In fact, we can run term-level queries with a constant score. They can be cached by the server, thus providing a performance benefit if the same query is rerun. These queries are like traditional database searches.

9.1.1 Term-level queries are not analyzed

One important characteristic of term-level queries is that they are not analyzed and tokenized (unlike full-text queries). The exception to this rule is when we use normalizers. Terms are matched against words stored in the inverted index without applying analyzers to match the indexing pattern. This means the search words must match the fields indexed in the inverted index.

For example, if we search for "Java" in a `title` field using a term-level query, chances are documents won't match. This is because during indexing, assuming we are using a `standard` analyzer, the word *Java* is converted to lowercase (*java*) and inserted into the title's inverted index. Because term-level queries are not analyzed, the engine tries to match the search word "Java" with the word "java" in the inverted index, so the match fails. We can return the same results from the query (with "Java" capitalized) if we use a `keyword` type instead (we explain shortly, so hang tight).

Term-level queries are suitable for keyword searches, not `text` field searches, because we know any field identified as a keyword is added to the inverted index without being analyzed during the indexing process. Like keywords, numerics, Booleans, ranges, and so on are not analyzed and are added directly to the respective inverted indexes.

9.1.2 Term-level query example

Let's take a simple example from the movie *The Godfather*. Figure 9.1 illustrates indexing and the term-level search. The `standard` analyzer doesn't find a hit because "The Godfather" doesn't exist as a single token stored in the inverted index (it is split into two tokens by the analyzer). Similarly, using just "Godfather" as a search word in the term-level query doesn't return any results because the word *Godfather* does not match the lowercased *godfather*.